Notiq Product Requirements Document (PRD)

1. Executive Summary

Product Name: Notiq
Version: 1.0 (MVP)
Date: December 2025
Team: Cícero (AI/Model), Carolina (Platform/Database/UI), Ishak (Project Manager)

Notiq is an AI-powered study platform that transforms traditional note-taking into an intelligent, interactive learning experience. By leveraging Google Gemini with RAG architecture, Notiq automatically structures, enhances, and contextualizes user notes while generating study materials like flashcards and quizzes.

Deployment: Vercel serverless platform

---

2. Product Vision & Goals

2.1 Vision Statement

Redefine notes as living study companions that help people learn faster, remember more, and feel confident in mastering knowledge.

2.2 Product Goals

- Reduce note preparation time by 60%
- Increase student engagement with structured study materials
- Provide personalized, context-aware learning assistance
- Support diverse input formats (text, PDF, handwritten notes)
- Maintain 95%+ accuracy in AI-generated content

2.3 Success Metrics

- User retention rate > 40% (30-day)
- Average session duration > 15 minutes
- Notes created per user > 10/month
- Flashcards/quizzes generated > 50/user/month
- User satisfaction score > 4.2/5.0

---

3. Target Audience

3.1 Primary Users

- **Students (Ages 16-25):** High school, undergraduate, and graduate students seeking efficient study tools
- **Characteristics:** Tech-savvy, time-constrained, seeking personalized learning solutions
- **Pain Points:** Disorganized notes, time-intensive preparation, difficulty retaining information

3.2 Secondary Users

- Professionals pursuing certifications
- Lifelong learners and autodidacts
- Educators creating study materials

---

4. Core Features & Requirements

4.1 Note Creation & Management

4.1.1 Input Methods

Priority: P0 (Must Have)

Requirements:

- Built-in markdown text editor with real-time preview
- PDF upload and parsing (max 50MB per file)
- Image upload for handwritten notes (JPEG, PNG, max 10MB)
- Support for drag-and-drop file uploads

Technical Specifications:

- **Backend:** Python serverless functions on Vercel
- **PDF Processing:** PyMuPDF (fitz) or pdfplumber
- **OCR:** Google Cloud Vision API
- **Storage:** Vercel Blob Storage or AWS S3
- **Database:** Vercel Postgres (serverless) + Supabase for vector storage

User Stories:

- As a student, I want to upload my lecture PDFs so they're automatically processed into structured notes
- As a user, I want to photograph my handwritten notes so they're digitized and searchable

4.1.2 Note Organization

Priority: P0

Requirements:

- Hierarchical folder structure (subjects > topics > subtopics)
- Tagging system with autocomplete
- Search functionality (full-text and semantic)
- Note versioning and history

Technical Specifications:

- Vercel Postgres with nested set model for hierarchy
- PostgreSQL full-text search with GIN indexes
- Supabase vector extension (pgvector) for semantic search

4.2 AI-Powered Note Structuring

4.2.1 Automatic Formatting

Priority: P0

Requirements:

- Convert raw text/uploads into standardized Notiq templates
- Extract and format headings, bullet points, definitions, examples
- Identify and structure key concepts, formulas, and important terms
- Generate table of contents for long notes

Technical Specifications:

- **LLM:** Google Gemini 2.0 Flash (gemini-2.0-flash-exp) via Google AI SDK
- **RAG Architecture:** Custom implementation with LangChain
- **Vector Store:** Supabase with pgvector extension

- **Embeddings:** text-embedding-004 (Google's embedding model)
- **Prompt Engineering:** Custom system prompts for consistent formatting

API Implementation (Python - Vercel Serverless):

```python
```

```python
# api/structure-note.py
from google import genai
from google.genai import types
import os
from supabase import create_client, Client

# Initialize clients
genai_client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))
supabase: Client = create_client(
    os.environ.get("SUPABASE_URL"),
    os.environ.get("SUPABASE_KEY")
)

def handler(request):
    """Vercel serverless function handler"""
    if request.method != 'POST':
        return {'statusCode': 405, 'body': 'Method not allowed'}

    data = request.get_json()
    raw_content = data.get('content')
    note_id = data.get('note_id')
    user_id = data.get('user_id')

    # Generate embedding for retrieval
    embedding_response = genai_client.models.embed_content(
        model='models/text-embedding-004',
        content=raw_content
    )
    query_embedding = embedding_response.embeddings[0].values

    # Retrieve relevant templates from vector store
    templates_response = supabase.rpc(
        'match_templates',
        {
            'query_embedding': query_embedding,
            'match_threshold': 0.75,
            'match_count': 3
        }
    ).execute()

    templates = templates_response.data
    template_context = "\n\n".join([t['content'] for t in templates])
```

```
    # Generate structured content with Gemini
    response = genai_client.models.generate_content(
        model='gemini-2.0-flash-exp',
        contents=[
            types.Content(
                role='user',
                parts=[
                    types.Part.from_text(f"""You are Notiq's note structuring AI. Transform the raw notes into a well-structured, stud

Use these template examples as reference:
{template_context}

Raw note content:
{raw_content}

Structure the note with:
1. Clear hierarchical headings
2. Bullet points for lists
3. **Bold** for key terms and concepts
4. Formatted formulas and equations
5. Examples in blockquotes
6. Summary section at the end

Return only the structured markdown content.""")
                ]
            )
        ],
        config=types.GenerateContentConfig(
            temperature=0.3,
            max_output_tokens=4096
        )
    )

    structured_content = response.text

    # Store structured note
    supabase.table('notes').update({
        'structured_content': structured_content,
        'embedding': query_embedding
    }).eq('id', note_id).execute()

    return {
        'statusCode': 200,
        'body': {
```

```
        'structured_content': structured_content,
        'note_id': note_id
    }
}
```

4.2.2 Content Enhancement

Priority: P1 (Should Have)

Requirements:

- Automatically add relevant diagrams/illustrations
- Suggest related concepts from user's note database
- Highlight ambiguous or incomplete information
- Generate summaries for each section

4.3 Flashcard Generation

4.3.1 Automatic Flashcard Creation

Priority: P0

Requirements:

- Generate question-answer flashcards from notes
- Support multiple flashcard types (basic, cloze, image-based)
- Allow manual editing of generated flashcards
- Organize flashcards by topic/subject

Technical Specifications:

- Gemini for Q&A pair extraction with structured output
- Spaced repetition algorithm (SM-2 or FSRS)
- Store flashcard metadata in Vercel Postgres
- Real-time generation via streaming responses

API Implementation:

```
python
```

```
# api/generate-flashcards.py
from google import genai
from google.genai import types
import json
import os

genai_client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))

def handler(request):
    data = request.get_json()
    note_content = data.get('content')
    note_id = data.get('note_id')
    difficulty = data.get('difficulty', 'medium')

    # Use Gemini with JSON schema for structured output
    response = genai_client.models.generate_content(
        model='gemini-2.0-flash-exp',
        contents=[
            types.Content(
                role='user',
                parts=[types.Part.from_text(f"""Generate flashcards from this note content.
Difficulty level: {difficulty}

Note content:
{note_content}

Create 10-15 flashcards in JSON format:
{{
  "flashcards": [
    {{
      "question": "clear question",
      "answer": "concise answer",
      "type": "basic",
      "difficulty": 1-5
    }}
  ]
}}""")]
            )
        ],
        config=types.GenerateContentConfig(
            temperature=0.4,
            response_mime_type='application/json',
            response_schema={
```

```
            'type': 'object',
            'properties': {
                'flashcards': {
                    'type': 'array',
                    'items': {
                        'type': 'object',
                        'properties': {
                            'question': {'type': 'string'},
                            'answer': {'type': 'string'},
                            'type': {'type': 'string'},
                            'difficulty': {'type': 'integer'}
                        }
                    }
                }
            }
        }
    )
)

flashcards_data = json.loads(response.text)

# Store in database
flashcards = []
for card in flashcards_data['flashcards']:
    result = supabase.table('flashcards').insert({
        'note_id': note_id,
        'question': card['question'],
        'answer': card['answer'],
        'card_type': card['type'],
        'difficulty': card['difficulty'],
        'next_review_date': datetime.now() + timedelta(days=1)
    }).execute()
    flashcards.append(result.data[0])

return {
    'statusCode': 200,
    'body': {'flashcards': flashcards}
}
```

User Stories:

- As a student, I want flashcards automatically generated from my notes so I can start studying immediately
- As a user, I want to edit AI-generated flashcards to match my understanding

4.3.2 Interactive Study Mode

Priority: P1

Requirements:

- Swipe-based flashcard interface (mobile-friendly)
- Confidence rating system (easy/medium/hard)
- Progress tracking and statistics
- Spaced repetition scheduling

4.4 Quiz Generation

4.4.1 Automatic Quiz Creation

Priority: P0

Requirements:

- Generate multiple-choice, true/false, and short-answer questions
- Difficulty level adjustment (easy/medium/hard)
- Time-limited quiz mode (optional)
- Instant feedback with explanations

Technical Specifications:

- Gemini-based question generation with structured JSON output
- Answer validation using semantic similarity
- Quiz state management in Vercel KV (Redis)

User Stories:

- As a student, I want to test my knowledge with auto-generated quizzes based on my notes
- As a user, I want detailed explanations for incorrect answers

4.4.2 Gamification Elements

Priority: P2 (Nice to Have)

Requirements:

- Points and achievement system
- Streak tracking
- Leaderboards (optional social feature)
- Progress badges

4.5 Q&A and Chatbot Interface

4.5.1 Contextual Q&A

Priority: P0

Requirements:

- Ask questions and receive answers based on user's notes
- Source attribution (show which notes contain the answer)
- Conversational follow-up questions
- Handle multi-document queries

Technical Specifications:

- RAG pipeline with Gemini and pgvector
- Hybrid search (semantic + keyword)
- Context window management (up to 1M tokens with Gemini 1.5 Pro)
- Citation tracking for source notes

API Implementation:

```
python
```

```python
# api/qa-chat.py
from google import genai
from google.genai import types
import os
from supabase import create_client

genai_client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))
supabase = create_client(
    os.environ.get("SUPABASE_URL"),
    os.environ.get("SUPABASE_KEY")
)

def handler(request):
    data = request.get_json()
    query = data.get('query')
    user_id = data.get('user_id')
    conversation_history = data.get('history', [])

    # Generate query embedding
    embedding_response = genai_client.models.embed_content(
        model='models/text-embedding-004',
        content=query
    )
    query_embedding = embedding_response.embeddings[0].values

    # Retrieve relevant note chunks using vector similarity
    relevant_notes = supabase.rpc(
        'match_notes',
        {
            'query_embedding': query_embedding,
            'match_threshold': 0.7,
            'match_count': 5,
            'user_id': user_id
        }
    ).execute()

    # Build context from retrieved notes
    context_parts = []
    sources = []

    for note in relevant_notes.data:
        context_parts.append(f"[Note: {note['title']}]\n{note['content']}\n")
        sources.append({
```

```
            'note_id': note['id'],
            'title': note['title'],
            'similarity': note['similarity']
        })

    context = "\n---\n".join(context_parts)

    # Build conversation with context
    contents = [
        types.Content(
            role='user',
            parts=[types.Part.from_text(f"""You are Notiq's Q&A assistant. Answer questions based ONLY on the user's notes pro

User's Notes Context:
{context}

Important guidelines:
1. Answer ONLY using information from the provided notes
2. If the answer isn't in the notes, say "I don't have that information in your notes"
3. Cite which note(s) you're referencing
4. Be concise but thorough

User question: {query}""")]
        )
    ]

    # Add conversation history if exists
    for msg in conversation_history[-5:]:  # Last 5 messages
        contents.append(
            types.Content(
                role=msg['role'],
                parts=[types.Part.from_text(msg['content'])]
            )
        )

    # Generate response with Gemini
    response = genai_client.models.generate_content(
        model='gemini-2.0-flash-exp',
        contents=contents,
        config=types.GenerateContentConfig(
            temperature=0.2,
            max_output_tokens=2048
        )
    )
```

```
answer = response.text

# Store conversation
supabase.table('conversations').insert({
    'user_id': user_id,
    'query': query,
    'answer': answer,
    'sources': sources
}).execute()

return {
    'statusCode': 200,
    'body': {
        'answer': answer,
        'sources': sources
    }
}
```

4.5.2 Chat History

Priority: P1

Requirements:

- Maintain conversation context across sessions
- Save important Q&A exchanges to notes
- Search through past conversations

4.6 User Interface & Experience

4.6.1 Design Principles

Priority: P0

Requirements:

- Clean, minimalist interface following "Clarity Over Complexity" value
- Responsive design (mobile, tablet, desktop)
- Dark mode support

- Accessibility compliance (WCAG 2.1 AA)

Technical Specifications:

- **Frontend Framework:** Next.js 14+ (for Vercel optimization)
- **UI Library:** Tailwind CSS + shadcn/ui
- **State Management:** React Context + SWR for data fetching
- **API Communication:** Native fetch with automatic retries

4.6.2 Key UI Components

- Dashboard with recent notes and study statistics
- Markdown editor with live preview
- Flashcard carousel with swipe gestures
- Quiz interface with timer and progress bar
- Chat interface with streaming responses

---

5. Technical Architecture

5.1 Backend Stack - Vercel Serverless

Framework: Python Serverless Functions on Vercel
Reasons:

- Zero-config deployment
- Automatic scaling
- Edge network optimization
- Excellent Next.js integration
- Built-in CI/CD

Project Structure:

```
notiq/
├── api/                    # Python serverless functions
│   ├── auth/
│   │   ├── register.py
│   │   └── login.py
│   ├── notes/
│   │   ├── create.py
│   │   ├── list.py
│   │   ├── upload.py
│   │   └── structure.py
│   ├── flashcards/
│   │   ├── generate.py
│   │   └── study.py
│   ├── quiz/
│   │   ├── generate.py
│   │   └── submit.py
│   └── qa/
│       ├── ask.py
│       └── history.py
├── app/                    # Next.js frontend
│   ├── (auth)/
│   ├── dashboard/
│   ├── notes/
│   └── study/
├── components/
├── lib/
├── public/
├── requirements.txt        # Python dependencies
└── vercel.json             # Vercel configuration
```

Core Dependencies (requirements.txt):

```
google-genai>=0.2.0
google-cloud-vision>=3.5.0
supabase>=2.0.0
PyMuPDF>=1.23.0
python-jose[cryptography]>=3.3.0
passlib[bcrypt]>=1.7.4
python-multipart>=0.0.6
Pillow>=10.0.0
pydantic>=2.5.0
```

vercel.json Configuration:

```json
{
  "functions": {
    "api/**/*.py": {
      "runtime": "python3.9",
      "maxDuration": 60,
      "memory": 3008
    }
  },
  "env": {
    "GEMINI_API_KEY": "@gemini-api-key",
    "SUPABASE_URL": "@supabase-url",
    "SUPABASE_KEY": "@supabase-key",
    "GOOGLE_CLOUD_VISION_KEY": "@gcp-vision-key"
  },
  "regions": ["iad1"]
}
```

5.2 Database Architecture

Primary Database: Vercel Postgres (Serverless)

- User accounts and authentication
- Note metadata (title, created_at, updated_at, user_id)
- Flashcard and quiz data
- User progress and statistics

Vector Database: Supabase with pgvector extension

- Note embeddings for semantic search (768 dimensions)
- RAG retrieval optimization
- Template storage
- Similarity search functions

Cache Layer: Vercel KV (Redis)

- Session management
- Rate limiting

- Temporary processing states
- API response caching

Object Storage: Vercel Blob Storage

- Uploaded PDFs and images
- Processed note files
- User-generated media
- Max 500MB per file on Hobby plan

Database Schema (SQL):

```sql
```

```sql
-- Vercel Postgres
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(255),
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),
    subscription_tier VARCHAR(50) DEFAULT 'free'
);

CREATE TABLE notes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(500) NOT NULL,
    content TEXT,
    structured_content TEXT,
    source_type VARCHAR(50),
    source_file_url TEXT,
    folder_id UUID,
    tags TEXT[],
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_notes_user ON notes(user_id);
CREATE INDEX idx_notes_tags ON notes USING GIN(tags);

-- Supabase with pgvector
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE note_embeddings (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL,
    note_id UUID NOT NULL,
    content TEXT NOT NULL,
    embedding vector(768),
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX ON note_embeddings USING ivfflat (embedding vector_cosine_ops);

-- Similarity search function
```

```sql
CREATE OR REPLACE FUNCTION match_notes(
    query_embedding vector(768),
    match_threshold float,
    match_count int,
    user_id uuid
)
RETURNS TABLE (
    id uuid,
    note_id uuid,
    title text,
    content text,
    similarity float
)
LANGUAGE sql STABLE
AS $$
    SELECT
        ne.id,
        ne.note_id,
        n.title,
        ne.content,
        1 - (ne.embedding <=> query_embedding) as similarity
    FROM note_embeddings ne
    JOIN notes n ON n.id = ne.note_id
    WHERE ne.user_id = match_notes.user_id
    AND 1 - (ne.embedding <=> query_embedding) > match_threshold
    ORDER BY similarity DESC
    LIMIT match_count;
$$;
```

5.3 AI/ML Pipeline with Google Gemini

5.3.1 RAG Architecture

```
User Query → Query Embedding (text-embedding-004) →
Vector Search (pgvector) → Context Retrieval →
Prompt Construction → Gemini 2.0 Flash →
Response Generation → Post-processing → User
```

5.3.2 Model Selection

Primary LLM: Google Gemini 2.0 Flash

- **Use Cases:** Note structuring, flashcard/quiz generation, Q&A

- **Advantages:**
  - 1M token context window
  - Native multimodal support (text, images, PDFs)
  - Structured JSON output
  - Lower cost than GPT-4
  - Fast inference
- **API:** Google AI Python SDK

Alternative for complex tasks: Gemini 1.5 Pro

- Use when deeper reasoning needed
- Longer context requirements

Embedding Model: text-embedding-004

- 768 dimensions
- Optimized for retrieval tasks
- Native Google integration

OCR: Google Cloud Vision API

- Document text detection
- Handwriting recognition
- Multi-language support

5.3.3 Gemini Features Utilized

Multimodal Input:

```
python
```

```python
# Process PDF directly with Gemini
with open('lecture.pdf', 'rb') as f:
    pdf_data = f.read()

response = genai_client.models.generate_content(
    model='gemini-2.0-flash-exp',
    contents=[
        types.Content(
            parts=[
                types.Part.from_bytes(
                    data=pdf_data,
                    mime_type='application/pdf'
                ),
                types.Part.from_text("Extract and structure the key concepts from this lecture PDF")
            ]
        )
    ]
)
```

Structured Output (JSON Mode):

```python
# Force JSON output with schema validation
response = genai_client.models.generate_content(
    model='gemini-2.0-flash-exp',
    contents=[...],
    config=types.GenerateContentConfig(
        response_mime_type='application/json',
        response_schema={
            'type': 'object',
            'properties': {
                'flashcards': {
                    'type': 'array',
                    'items': {'type': 'object'}
                }
            }
        }
    )
)
```

Streaming Responses:

```python
# For chat interface
for chunk in genai_client.models.generate_content_stream(
    model='gemini-2.0-flash-exp',
    contents=[...]
):
    yield chunk.text
```

5.3.4 Prompt Engineering Strategy

- Maintain prompt library in `/lib/prompts/`
- Use few-shot examples for consistency
- Implement prompt templates with Jinja2
- Version control all prompts
- A/B test prompt variations

Example Prompt Template:

```python
# lib/prompts/structure_note.py
STRUCTURE_NOTE_PROMPT = """You are Notiq's note structuring AI. Your job is to transform raw student notes into we

RULES:
1. Use clear hierarchical headings (# ## ###)
2. Bold **key terms** and concepts
3. Use bullet points for lists
4. Format mathematical equations properly
5. Add a summary at the end
6. Keep the original meaning intact

TEMPLATE EXAMPLES:
{template_examples}

RAW NOTE:
{raw_content}

OUTPUT:
Return ONLY the structured markdown content."""
```

5.4 API Design

API Style: RESTful with serverless functions

Authentication: JWT (JSON Web Tokens)

- Access token (15 min expiry)
- Refresh token (7 day expiry)
- Stored in Vercel KV

Key Endpoints:

```
Authentication:
POST   /api/auth/register
POST   /api/auth/login
POST   /api/auth/refresh

Notes:
GET    /api/notes/list
POST   /api/notes/create
GET    /api/notes/[id]
PUT    /api/notes/[id]/update
DELETE /api/notes/[id]
POST   /api/notes/upload
POST   /api/notes/[id]/structure

Flashcards:
GET    /api/flashcards/list
POST   /api/flashcards/generate
PUT    /api/flashcards/[id]/update
POST   /api/flashcards/study-session

Quizzes:
POST   /api/quiz/generate
GET    /api/quiz/[id]
POST   /api/quiz/[id]/submit

Q&A:
POST   /api/qa/ask
GET    /api/qa/history
```

Example Serverless Function:

python

```python
# api/notes/create.py
from http.server import BaseHTTPRequestHandler
import json
import os
from supabase import create_client
from google import genai
import jwt
from datetime import datetime

supabase = create_client(
    os.environ['SUPABASE_URL'],
    os.environ['SUPABASE_KEY']
)

genai_client = genai.Client(api_key=os.environ['GEMINI_API_KEY'])

class handler(BaseHTTPRequestHandler):
    def do_POST(self):
        # Verify JWT token
        auth_header = self.headers.get('Authorization')
        if not auth_header:
            self.send_error(401, 'Unauthorized')
            return

        try:
            token = auth_header.split(' ')[1]
            payload = jwt.decode(
                token,
                os.environ['JWT_SECRET'],
                algorithms=['HS256']
            )
            user_id = payload['user_id']
        except:
            self.send_error(401, 'Invalid token')
            return

        # Parse request body
        content_length = int(self.headers['Content-Length'])
        body = self.rfile.read(content_length)
        data = json.loads(body)

        title = data.get('title')
        content = data.get('content')
```

```
    # Create note in database
    result = supabase.table('notes').insert({
        'user_id': user_id,
        'title': title,
        'content': content,
        'source_type': 'manual',
        'created_at': datetime.now().isoformat()
    }).execute()

    note = result.data[0]

    # Generate embedding asynchronously (background task)
    # This will be called separately

    self.send_response(200)
    self.send_header('Content-type', 'application/json')
    self.end_headers()
    self.wfile.write(json.dumps({
        'success': True,
        'note': note
    }).encode())
```

5.5 Background Job Processing

Challenge: Vercel serverless functions have 60s timeout (Pro: 300s)

Solutions:

1. **Vercel Cron Jobs** for scheduled tasks
2. **Queue system** with Vercel KV + separate worker
3. **Webhook patterns** for long-running tasks
4. **Streaming responses** for real-time feedback

Implementation Example:

python

```python
# For PDF processing that might take >60s
# 1. Return immediately with job ID
# 2. Process asynchronously
# 3. Notify via webhook or polling

# api/notes/upload.py
def handler(request):
    # Upload file to Vercel Blob
    blob_url = upload_to_blob(file)

    # Create job entry
    job_id = create_processing_job(blob_url, user_id)

    # Trigger async processing (separate endpoint)
    trigger_async_processing(job_id)

    return {
        'statusCode': 202,
        'body': {
            'job_id': job_id,
            'status': 'processing',
            'webhook_url': f'/api/jobs/{job_id}/status'
        }
    }

# api/jobs/process.py - Called by queue worker
def process_pdf(job_id):
    # Long-running PDF processing
    # No timeout restrictions here
    pass
```

---

6. Data Models

6.1 Core Entities

User

python

```python
from pydantic import BaseModel, EmailStr
from uuid import UUID
from datetime import datetime

class User(BaseModel):
    id: UUID
    email: EmailStr
    password_hash: str
    full_name: str
    created_at: datetime
    updated_at: datetime
    subscription_tier: str = "free"
    preferences: dict = {}
```

Note

```python
class Note(BaseModel):
    id: UUID
    user_id: UUID
    title: str
    content: str
    structured_content: str | None = None
    source_type: str  # manual, pdf, image
    source_file_url: str | None = None
    folder_id: UUID | None = None
    tags: list[str] = []
    created_at: datetime
    updated_at: datetime
```

Flashcard

```python
```

```python
class Flashcard(BaseModel):
    id: UUID
    note_id: UUID
    user_id: UUID
    question: str
    answer: str
    card_type: str  # basic, cloze, image
    difficulty: int  # 1-5
    next_review_date: datetime
    review_count: int = 0
    ease_factor: float = 2.5  # For SM-2 algorithm
    created_at: datetime
```

Quiz

python

```python
class QuizQuestion(BaseModel):
    question: str
    options: list[str]
    correct_answer: str | int
    explanation: str
    question_type: str  # multiple_choice, true_false, short_answer

class Quiz(BaseModel):
    id: UUID
    note_id: UUID | None
    user_id: UUID
    title: str
    questions: list[QuizQuestion]
    difficulty: str
    time_limit: int | None
    created_at: datetime

class QuizAttempt(BaseModel):
    id: UUID
    quiz_id: UUID
    user_id: UUID
    score: float
    answers: dict
    completed_at: datetime
```

---

7. Non-Functional Requirements

7.1 Performance

- API response time < 200ms (p95) for standard requests
- Gemini response time < 3s for note structuring
- Support 1000 concurrent users (MVP)
- File upload processing < 30s for 10MB files
- Vercel Edge Network optimization for global latency < 100ms

7.2 Scalability

- Automatic scaling via Vercel serverless (0 to infinity)
- Vercel Postgres connection pooling (handled automatically)
- CDN for static assets via Vercel Edge
- Supabase auto-scaling for vector operations

7.3 Security

- HTTPS only (TLS 1.3) - automatic with Vercel
- Input validation and sanitization
- Rate limiting via Vercel KV (100 req/min per user)
- SQL injection prevention via Supabase client
- XSS protection
- CORS configuration
- Environment variables via Vercel secrets

7.4 Reliability

- 99.9% uptime target (Vercel SLA)
- Automated backups via Vercel Postgres
- Error logging with Vercel Analytics
- Graceful degradation if Gemini API is down
- Automatic retries for transient failures

7.5 Privacy & Compliance

- GDPR compliance for EU users
- User data encryption at rest (AES-256)
- Encryption in transit (TLS)
- Right to deletion implementation
- Clear data retention policies (3 years inactive accounts)

---

8. Technical Challenges & Mitigations

8.1 Gemini Hallucination Prevention

Challenge: AI generating incorrect or synthetic information

Mitigations:

- Strict RAG with high similarity thresholds (> 0.7)
- System prompts emphasizing "only use provided context"
- Grounding with Google Search (optional Gemini feature)
- User feedback loop to flag incorrect outputs
- Confidence scores on generated content
- Citation enforcement in prompts

Example Prompt Pattern:

```
python

GROUNDING_PROMPT = """CRITICAL INSTRUCTIONS:
1. Answer ONLY using information from the provided notes below
2. If information is not in the notes, respond: "I don't have that information in your notes"
3. Do NOT use your general knowledge
4. Cite specific notes when answering

Notes Context:
{context}

Question: {query}"""
```

8.2 Vercel Serverless Timeout Limitations

Challenge: 60s timeout for Hobby, 300s for Pro

Mitigations:

- Chunk large operations into smaller tasks
- Use async job queue pattern for PDF processing
- Implement progress webhooks
- Stream responses for long-running tasks
- Use Vercel Cron for scheduled heavy operations
- Upgrade to Pro plan for critical operations ($20/month)

8.3 Multi-Format Data Integration

Challenge: Processing diverse input formats consistently

Mitigations:

- Leverage Gemini's native multimodal capabilities (PDFs, images)
- Standardize all inputs to markdown intermediate format
- Robust error handling for malformed files
- Google Cloud Vision fallback for OCR
- User preview before final processing

Gemini Multimodal Advantage:

```python
# Gemini can process PDFs directly - no parsing needed!
response = genai_client.models.generate_content(
    model='gemini-2.0-flash-exp',
    contents=[
        types.Part.from_bytes(pdf_bytes, mime_type='application/pdf'),
        types.Part.from_text("Structure this lecture content")
    ]
)
```

8.4 Context Window Management

Challenge: Managing large note collections

Mitigations:

- Gemini 1.5 Pro: 1M token context (huge advantage!)
- Smart chunking strategy (500-1000 tokens per chunk)
- Hybrid search (semantic + keyword)
- Query decomposition for complex questions
- Context compression via summarization
- Use text-embedding-004 for efficient retrieval

8.5 Cost Optimization

Challenge: API costs with Gemini

Mitigations:

- Use Gemini 2.0 Flash (cheaper than Pro) for most operations
- Cache common queries in Vercel KV
- Batch embeddings generation
- Rate limiting per user
- Implement usage quotas

Pricing Reference (Gemini):

- Gemini 2.0 Flash: $0.075 / 1M input tokens, $0.30 / 1M output
- Gemini 1.5 Pro: $1.25 / 1M input tokens, $5.00 / 1M output
- text-embedding-004: $0.00025 / 1K tokens

8.6 Cold Start Issues

Challenge: Serverless cold starts affecting performance

Mitigations:

- Keep functions warm with Vercel Cron (ping every 5 min)
- Optimize function size (reduce dependencies)
- Use Vercel Edge Functions for critical paths
- Cache responses in Vercel KV
- Consider Pro plan (50% faster cold starts)

9. Development Roadmap

Phase 1: MVP (Months 1-3)

P0 Features:

- User authentication and basic profile
- Manual note creation (markdown editor)
- PDF upload with Gemini direct processing
- Simple note structuring via Gemini
- Basic flashcard generation
- Q&A chatbot with RAG
- Simple dashboard UI
- Deploy to Vercel

Deliverables:

- Next.js application
- Python API functions
- Vercel Postgres + Supabase setup
- Gemini integration
- Working frontend

Technical Setup:

```bash
```

```
# Initialize project
npx create-next-app@latest notiq --typescript --tailwind --app
cd notiq

# Setup Python API
mkdir api
pip install -r requirements.txt

# Deploy to Vercel
vercel login
vercel --prod
```

Phase 2: Enhancement (Months 4-5)

P1 Features:

- Image upload with OCR (Google Cloud Vision)
- Advanced note organization (folders, tags)
- Quiz generation with explanations
- Spaced repetition algorithm
- Chat history and saved Q&As
- Mobile-responsive design
- Dark mode
- Streaming responses

Phase 3: Optimization (Month 6)

P2 Features:

- Gamification elements
- Social features (optional)
- Advanced analytics dashboard
- Export functionality (PDF, markdown)
- Performance tuning
- Cost optimization
- Production monitoring

10. Deployment & DevOps

10.1 Vercel Deployment

Automatic Deployment:

yaml

```yaml
# .github/workflows/deploy.yml
name: Deploy to Vercel
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - uses: vercel/actions-deploy@v2
        with:
          vercel-token: ${{ secrets.VERCEL_TOKEN }}
          vercel-org-id: ${{ secrets.VERCEL_ORG_ID }}
          vercel-project-id: ${{ secrets.VERCEL_PROJECT_ID }}
```

Environment Variables (Vercel Dashboard):

```
GEMINI_API_KEY=your_gemini_key
SUPABASE_URL=your_supabase_url
SUPABASE_KEY=your_supabase_anon_key
GOOGLE_CLOUD_VISION_KEY=your_gcp_key
JWT_SECRET=your_jwt_secret
NEXT_PUBLIC_API_URL=https://notiq.vercel.app
```

10.2 Database Setup

Vercel Postgres:

bash

```
# Create database via Vercel dashboard
vercel postgres create

# Connect locally
vercel postgres connect

# Run migrations
psql $POSTGRES_URL -f schema.sql
```

Supabase Setup:

```
sql

-- Enable pgvector extension
CREATE EXTENSION vector;

-- Create tables and functions
-- (see section 5.2 for full schema)
```

10.3 Monitoring

Vercel Analytics:

- Automatic page view tracking
- Web Vitals monitoring
- Function execution metrics

Custom Logging:

```
python

# api/lib/logger.py
import logging
import os

logger = logging.getLogger('notiq')
logger.setLevel(logging.INFO)

def log_api_call(endpoint, duration, user_id=None):
    logger.info(f"API: {endpoint} | Duration: {duration}ms | User: {user_id}")
```

---

11. Testing Strategy

11.1 Unit Tests

python

```python
# tests/test_gemini_integration.py
import pytest
from api.lib.gemini_client import structure_note

def test_note_structuring():
    raw_content = "Introduction to calculus. Limits and derivatives."
    result = structure_note(raw_content)

    assert result is not None
    assert "# " in result  # Has headings
    assert len(result) > len(raw_content)  # Enhanced content

def test_flashcard_generation():
    note_content = "The mitochondria is the powerhouse of the cell."
    flashcards = generate_flashcards(note_content)

    assert len(flashcards) > 0
    assert all('question' in card for card in flashcards)
    assert all('answer' in card for card in flashcards)
```

11.2 Integration Tests

python

```python
# tests/test_api_endpoints.py
import pytest
from fastapi.testclient import TestClient

def test_create_note(client, auth_token):
    response = client.post(
        "/api/notes/create",
        json={"title": "Test Note", "content": "Test content"},
        headers={"Authorization": f"Bearer {auth_token}"}
    )
    assert response.status_code == 200
    assert response.json()['note']['title'] == "Test Note"

def test_qa_endpoint(client, auth_token):
    response = client.post(
        "/api/qa/ask",
        json={"query": "What is calculus?"},
        headers={"Authorization": f"Bearer {auth_token}"}
    )
    assert response.status_code == 200
    assert 'answer' in response.json()
```

11.3 Gemini Testing

- Prompt regression testing
- Response quality validation
- Hallucination detection tests
- Cost tracking per operation

11.4 E2E Tests

typescript

```
// tests/e2e/note-creation.spec.ts
import { test, expect } from '@playwright/test';

test('user can create and structure a note', async ({ page }) => {
  await page.goto('/dashboard');
  await page.click('text=New Note');
  await page.fill('[name="title"]', 'Biology Notes');
  await page.fill('[name="content"]', 'Cell structure and function');
  await page.click('text=Structure with AI');

  await expect(page.locator('.structured-note')).toBeVisible();
});
```

---

12. Cost Estimation

12.1 Vercel Costs

- **Hobby Plan:** $0/month (good for MVP)
  - 100GB bandwidth
  - 100GB-hours serverless function execution
  - 60s function timeout
- **Pro Plan:** $20/month (recommended for production)
  - 1TB bandwidth
  - 1000GB-hours execution
  - 300s function timeout

12.2 Gemini API Costs

Assumptions: 100 active users/month

Operation	Volume/User	Cost/1K tokens	Monthly Cost
Note structuring	20 notes	$0.075 in, $0.30 out	~$15
Flashcard generation	50 sets	$0.075 in, $0.30 out	~$20
Q&A queries	100 queries	$0.075 in, $0.30 out	~$25
Embeddings	200 notes	$0.00025	~$2

Total Gemini: ~$62/month for 100 users = **$0.62/user/month**

12.3 Database Costs

- **Vercel Postgres:** Included in Pro plan or $10/month standalone
- **Supabase:** $25/month (Pro tier for vector operations)
- **Vercel Blob:** $0.15/GB stored, $0.10/GB bandwidth

Total Infrastructure: ~$55-75/month

---

13. Success Criteria

13.1 MVP Launch Criteria

- [ ] 100+ beta users successfully onboarded
- [ ] < 5% error rate on core features
- [ ] 99.5% uptime over 30-day period
- [ ] Positive user feedback (>3.5/5 average)
- [ ] All P0 features functional
- [ ] Gemini API costs < $1/user/month

13.2 Product-Market Fit Indicators

- 40%+ monthly active user retention
- 20+ notes created per active user/month
- NPS score > 30
- Organic user growth rate > 10%/month
- Feature request pipeline from users
- Gemini response quality > 90% satisfaction

---

14. Risk Mitigation

14.1 Gemini API Risks

Risk: API outages or rate limits

Mitigation:

- Implement fallback to cached responses
- Queue system for non-urgent requests

- Clear user communication during outages
- Consider multi-provider strategy (add Claude as backup)

14.2 Vercel Free Tier Limits

Risk: Exceeding free tier during growth

Mitigation:

- Monitor usage dashboard daily
- Set up alerts at 80% quota
- Upgrade to Pro before hitting limits
- Implement aggressive caching

14.3 Cost Overruns

Risk: Unexpectedly high Gemini costs

Mitigation:

- Per-user rate limiting (20 AI operations/day free tier)
- Implement usage quotas
- Cache common responses
- Monitor costs in real-time
- Premium tier for power users

---

15. Appendix

15.1 Quick Start Guide

Local Development:

```bash
```

```
# Clone repository
git clone https://github.com/your-org/notiq.git
cd notiq

# Install dependencies
npm install
pip install -r requirements.txt

# Setup environment
cp .env.example .env.local
# Add your API keys

# Run development server
npm run dev

# In another terminal, test Python functions
vercel dev
```

15.2 Useful Resources

- Gemini API Docs: https://ai.google.dev/docs
- Vercel Python Runtime: https://vercel.com/docs/functions/runtimes/python
- Supabase pgvector: https://supabase.com/docs/guides/ai
- Next.js on Vercel: https://vercel.com/docs/frameworks/nextjs

15.3 Team Contacts

- Project Manager: Ishak
- AI/Model Lead: Cícero (Gemini integration)
- Platform Lead: Carolina (Vercel deployment, database)

Document Version: 2.0 (Gemini + Vercel)
Last Updated: December 2025
Next Review: January 2026